



NAS Data Management Platform

AI-Powered File Operations Command Center

Graduation Project 1 — Phase 1 MVP Technical Report

Executive Positioning

This report presents the NAS Data Management Platform as a working, scalable, and safety-aware MVP for intelligent file management inside Network Attached Storage environments. It is written to communicate project value clearly to technical evaluators, a audience, and the university committee.

University	Jordan University of Science and Technology
College	College of Computer Sciences & Information Technology
Department	Software Engineering
Supervisor	Dr. Yanal Nawaf Saleh Alahmad.
Team	

April 2026

Executive Summary for Review

NAS Data Management Platform is presented as a practical product concept, not only an academic prototype. The Phase 1 MVP proves the most important technical question: can a user interact with NAS files through a modern web dashboard, receive AI-assisted responses, and remain protected by a confirmation layer before operations? The implemented direction answers yes by combining a React command center, Node.js API, SMB/NAS file service, AI assistant, and visible operation feedback.

The platform is valuable because file chaos is a real operational problem. Teams lose time searching for files, storage is wasted by duplicated or poorly named assets, and manual operations increase risk. This system introduces an intelligent layer above the NAS that can evolve into a production-grade internal tool for companies managing large creative, technical, or administrative file repositories.

Dimension	Level Message
Strategic Value	Transforms a passive NAS file browser into an intelligent operations center.
Technical Proof	Demonstrates frontend, backend, file-service, AI-service, and confirmation workflow integration.
Risk Control	Uses preview and confirmation before operations to reduce accidental file damage.
Scalability Direction	Prepared for GP2 features: duplicate detection, rollback, audit logs, active-file protection, and multi-NAS.

Phase 1 Compliance Snapshot

The MVP covers the required Phase 1 direction: dashboard frontend, SMB/NAS connection layer, list files feature, AI chat integration, confirmation dialog, clean structure, screenshots, UML, and testing evidence.

Undertaking

This is to declare that the project entitled “NAS Data Management Platform” is an original work completed by the project team in partial fulfillment of the requirements for the Bachelor degree in Software Engineering at the Software Engineering Department, College of Computer Sciences and Information Technology, Jordan University of Science and Technology.

All analysis, design, implementation, testing, documentation, and presentation materials described in this report are aligned with the submitted Graduation Project 1 scope and Phase 1 MVP requirements. The work presented here is intended to serve as the technical foundation for the continued implementation in Graduation Project 2.

Student signatures:

Student 1: _____ Signature: _____

Student 2: _____ Signature: _____

Student 3: _____ Signature: _____

Abstract

NAS Data Management Platform is an AI-powered web application for managing and organizing files stored in Network Attached Storage environments. The GP1 phase delivers a working MVP that connects a React dashboard with a Node.js backend, supports SMB/NAS file listing, provides AI-assisted file commands, displays file metadata, and introduces confirmation dialogs before sensitive operations. The system addresses chaotic file naming, manual file browsing, limited visibility, and unsafe bulk actions. It establishes a clean foundation for GP2 by separating the user interface, backend API, SMB service, AI service, and operation logging. The result is a practical command center that demonstrates technical feasibility, user value, and future scalability for real organizational file management.

Acknowledgment

The project team would like to express appreciation to the project supervisor for continuous guidance, evaluation feedback, and support throughout the planning and implementation of Graduation Project 1. We also acknowledge Jordan University of Science and Technology and the College of Computer Sciences and Information Technology for providing the academic framework that guided the project's requirements, design, testing, and documentation. Finally, we appreciate the support of all individuals who contributed feedback during the development of the MVP.

Table of Contents

Chapter 1: Project Overview, Vision, and Planning

- 1.1 Problem Statement
- 1.2 Related Products
- 1.3 Product Vision
- 1.4 Project Objectives and Milestones
- 1.5 Risk Assessment and Mitigation

Chapter 2: Product Features and Requirements

- 2.1 Functional Features
- 2.2 Feature-to-Requirement Mapping

Chapter 3: System Design and Deployment Overview

- 3.1 System Architecture
- 3.2 Detailed Design and UML Models
- 3.3 Software Deployment

Chapter 4: System Development and Implementation

- 4.1 Core Implementation Progress
- 4.2 Implemented and Planned Features
- 4.3 Screenshots Evidence

Chapter 5: Testing

- 5.1 Testing Overview
- 5.2 Sample Test Cases
- 5.3 Test Reports

References

CHAPTER 1: Project Overview, Vision, and Planning

1.1 Problem Statement

Organizations that rely on NAS servers often accumulate thousands of files across design, media, development, and administrative folders. In real environments, files are created by different team members, saved under inconsistent names, copied between folders, forgotten after delivery, and stored without a clear operational view. Examples such as “final_final_v2.mp4”, “untitled_123.psd”, or repeated exported files are not small cosmetic issues; they create search delays, duplicated storage cost, project confusion, and risk during handover.

Traditional file managers display files, but they rarely explain what is wrong with the data. They do not identify risky names, do not understand natural-language file requests, and do not protect users with clear previews before operations. As a result, teams still depend on manual browsing, manual renaming, and informal knowledge about where assets are stored. This problem becomes more serious for creative teams and IT administrators who handle large volumes of media files, where one wrong rename or delete operation can affect an active project.

The NAS Data Management Platform addresses this problem by turning file management into an intelligent command center. Instead of simply showing folders, the system scans, lists, interprets commands, highlights risks, and prepares safe operations with confirmation before execution. Phase 1 focuses on proving the foundation: dashboard, file listing, SMB/NAS integration, AI chat, confirmation dialog, and clean documented code.

Why this project matters

The value is not only “showing files.” The value is giving organizations visibility, safer operations, AI-assisted interaction, and a scalable architecture that can become a production file-management product.

1.2 Related Products

Related Product	Strength	Gap Addressed by Our System
Windows File Explorer / macOS Finder	Simple file browsing, copy, rename, delete.	No AI assistant, no centralized NAS intelligence, no project-specific rules, no smart risk insights.
Synology / QNAP File Station	Strong NAS browser with user access and file operations.	Mostly platform-specific; limited AI command understanding and limited customizable naming intelligence for creative workflows.
Nextcloud / ownCloud	Collaboration, file sharing, sync, web access.	Cloud-collaboration focus rather than NAS operation command center and AI-driven file quality management.
Enterprise DMS Tools	Governance, document workflow, permissions, retention.	Often heavy, expensive, document-centered, and not optimized for simple GP1 MVP deployment with SMB and AI assistant.

Our system differentiates itself by combining a practical web dashboard, SMB/NAS file access, AI-powered command interpretation, file quality insights, and safety confirmation. The goal is to bridge the gap between simple file browsing and enterprise-grade data management without overcomplicating the first MVP.

1.3 Product Vision

The proposed solution is a web-based NAS command center that helps users view files and folders, inspect file metadata, ask the AI assistant for file-related actions, and execute safe file operations through confirmation prompts. In GP1, the system focuses on the minimum viable foundation: a React frontend, Node.js backend, SMB/NAS connection layer, file listing, AI chat integration, and basic operation confirmation.

Target users include IT administrators who manage NAS infrastructure, creative teams that work with design/video/audio assets, software project teams that store structured project files, and organizations that want better visibility over large internal storage. The product vision statement is: Our system helps teams managing NAS-based files reduce manual effort and operational risk by providing an intelligent, safe, and extensible file operations command center.

The value proposition is practical: the platform does not replace the NAS; it improves how humans interact with it. It introduces AI-assisted search and command interpretation, clearer file metadata, risk-oriented insights, and a clean architecture that can later support duplicate detection, naming rules, archiving policies, rollback, multi-NAS management, and richer audit logs.

Scope of GP1

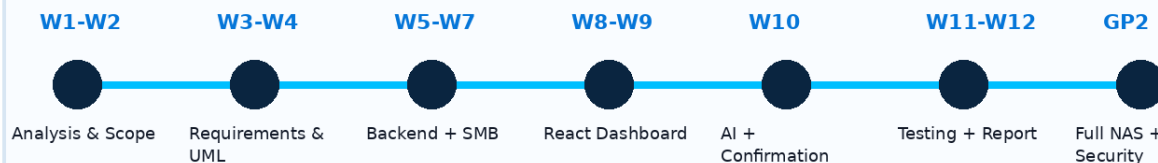
Included: dashboard frontend, file table, file details, backend API, SMB/NAS connection or documented mock mode, AI chat, confirmation dialog, screenshots, test cases, UML, and report. Not included in GP1: full authentication, production credential vault, real destructive operations, enterprise RBAC, and complete multi-NAS production deployment.

1.4 Project Objectives and Milestones

Objective ID	Objective
O1	Build a working web dashboard that loads without errors and presents the platform professionally.
O2	Implement backend APIs for health check, file listing, AI chat, and operation confirmation support.
O3	Connect to NAS through SMB protocol or demonstrate a documented mock/fallback mode for Phase 1.
O4	Display file and folder metadata including name, type, size, path, status, and modified date.
O5	Integrate AI chat to answer basic file-related queries such as "Show files in /Work".
O6	Provide confirmation dialog before file operations and record activity in a visible log.
O7	Prepare UML diagrams, screenshots, test cases, documentation, and a clean final report.

The following roadmap defines what was targeted in GP1 and what will be extended in GP2:

GP1 to GP2 Roadmap



Milestone	Description	Start	End	Deliverable
M1	Problem analysis and scope definition	Week 1	Week 2	Problem statement and vision
M2	Requirements and feature mapping	Week 2	Week 3	Functional and non-functional requirements
M3	Backend foundation and API setup	Week 4	Week 5	Express API and health endpoint
M4	SMB/NAS listing service	Week 5	Week 7	File listing endpoint and metadata mapping
M5	React dashboard implementation	Week 7	Week 9	Command center UI and file explorer
M6	AI chat and confirmation dialog	Week 9	Week 10	AI assistant and safe operation preview

M7	Testing, UML, screenshots, report	Week 11	Week 12	Final GP1 submission package
GP2	Full implementation expansion	Next course	End of GP2	Security, rollback, audit, multi-NAS, advanced rules

1.5 Risk Assessment and Mitigation

Risk ID	Description	Impact	Mitigation Strategy
R1	SMB access may not be available during demo or from the development network.	High	Implement a documented mock mode that returns realistic file metadata while keeping the SMB service interface unchanged.
R2	AI API failure, rate limit, or missing key may interrupt the chat demo.	Medium	Provide graceful fallback messages and allow file listing to remain functional without AI.
R3	Unsafe file operations could damage real files if executed without review.	High	Restrict GP1 operations to preview/confirmation and avoid destructive changes; add clear confirmation modal.
R4	UI may look like a simple assignment instead of a product.	Medium	Use a command-center dashboard with status badges, insights, activity log, and professional visual hierarchy.
R5	Team members may not be able to explain implementation ownership.	High	Maintain clean code, README files, diagrams matching the code, and a demo script for each implemented feature.
R6	Scope creep could delay GP1 deliverables.	Medium	Separate GP1 MVP from GP2 advanced features and keep traceability tables aligned with Phase 1 criteria.

CHAPTER 2: Product Features and Requirements

2.1 Functional Features

The GP1 feature set was selected to demonstrate the character of the final product while remaining feasible within Phase 1. The key principle is that the MVP must be a foundation for GP2, not a disposable prototype. Each feature below is visible to the user and connected to an implemented or planned requirement.

Feature ID	Feature Name	Description	Priority	Stage
F1	Dashboard Command Center	Main web interface showing status badges, summary cards, file explorer, AI assistant, and activity log.	High	GP1 Implemented
F2	SMB/NAS Connection	Backend service connects to NAS using SMB2 or documented mock mode for demo continuity.	High	GP1 Implemented / Mock-ready
F3	List Files and Folders	Display files from a selected path with name, type, size, path, date, and status.	High	GP1 Implemented
F4	AI Chat Assistant	Accept file-related natural-language prompts and return contextual responses.	High	GP1 Implemented
F5	Confirmation Dialog	Display operation details and require user approval before file operations.	High	GP1 Implemented
F6	File Risk Insight	Identify suspicious naming such as repeated “final” or unclear versions and show recommended action.	Medium	GP1 Partial
F7	Activity Log	Record visible actions such as backend connection, file load, AI command, and confirmation result.	Medium	GP1 Implemented
F8	Reports and Audit Expansion	Generate structured reports and detailed audit history for operations.	Medium	GP2 Planned
F9	Advanced Naming Standards	Configurable naming rules, suggestion/enforcement modes, conflict handling.	Medium	GP2 Planned
F10	Rollback and Multi-NAS	Undo recent operations and support multiple NAS endpoints.	Medium	GP2 Planned

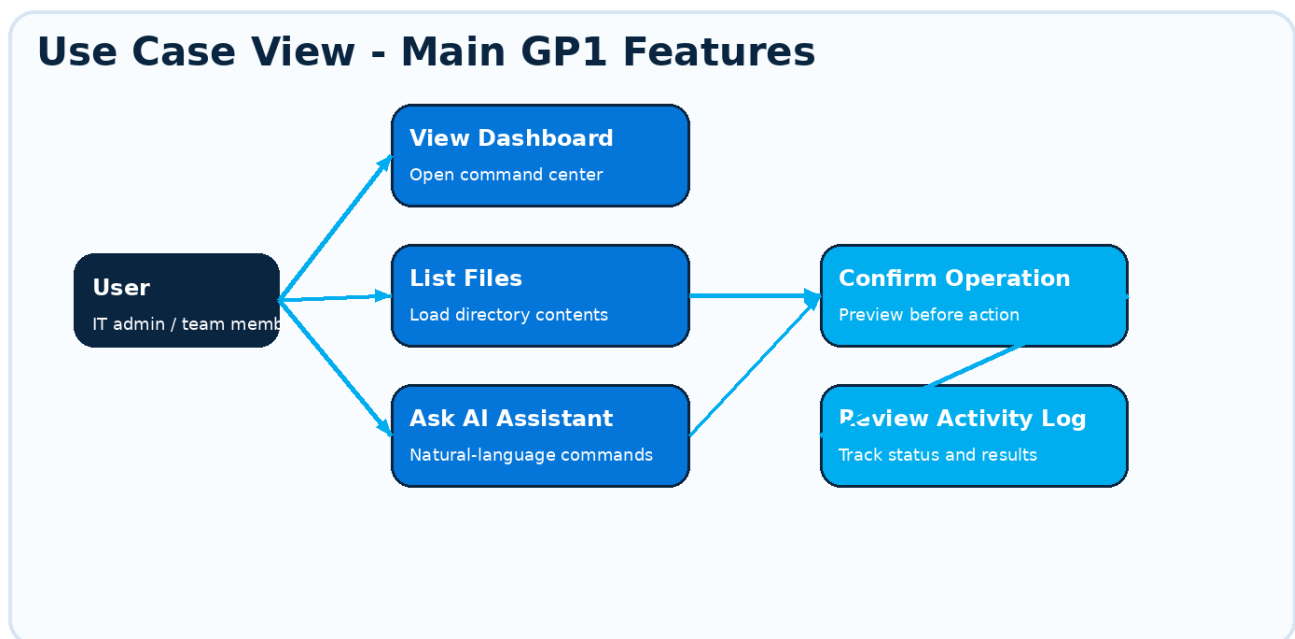


Figure 1. Use case view of the main GP1 features.

2.2 Feature-to-Requirement Mapping

Feature	Requirement ID	Requirement Description	Priority
F1	FR-01	The system shall display a dashboard with platform title, connection status, NAS mode, AI status, storage summary, and navigation.	High
F1	NFR-01	The dashboard should load within three seconds in a local development environment.	Medium
F2	FR-02	The backend shall expose an SMB/NAS service capable of reading a specified directory path.	High
F2	NFR-02	The system shall handle NAS connection failure without crashing the frontend.	High
F3	FR-03	The system shall list files and folders from a selected directory.	High
F3	FR-04	The system shall display file metadata: name, type, size, path, modified date, and status.	High
F4	FR-05	The system shall allow the user to submit file-related messages to the AI assistant.	High
F4	FR-06	The AI assistant shall respond to basic file queries such as “Show files in /Work”.	High
F4	NFR-03	AI failures shall return a friendly fallback message to the user.	Medium
F5	FR-07	The system shall show a confirmation dialog before executing or simulating a file operation.	High
F5	FR-08	The confirmation dialog shall display operation name, target path, and expected effect.	High
F6	FR-09	The system shall detect at least one risky naming pattern such as “final_final” in GP1 sample data.	Medium
F7	FR-10	The system shall record visible activity events for major user actions.	Medium
F8	NFR-04	Reports and audit logs should be expandable in GP2 without redesigning the backend.	Medium
F9	NFR-05	Naming standards shall be configurable in GP2 rather than hardcoded.	Medium
F10	NFR-06	Future file operations shall support rollback and safe recovery policies.	Medium

Requirement Collection Techniques

The requirements were collected from the Phase 1 project specification, university GP1 report template, evaluation rubric, stakeholder-style project brief, and analysis of the prototype dashboard screen. The resulting requirements were kept measurable, testable, and aligned with the MVP deliverables.

CHAPTER 3: System Design and Deployment

Overview

3.1 System Architecture

The system follows a layered architecture that separates the React user interface from the Node.js backend and separates backend routes from service-level responsibilities. This separation makes the system easier to test, explain, and extend in GP2. The frontend is responsible for displaying files, chat messages, status indicators, confirmations, and activity feedback. The backend exposes API endpoints, validates requests, calls the SMB service for file access, and calls the AI service for natural-language responses. This design supports mock mode in GP1 while preserving a clean path toward real NAS integration.

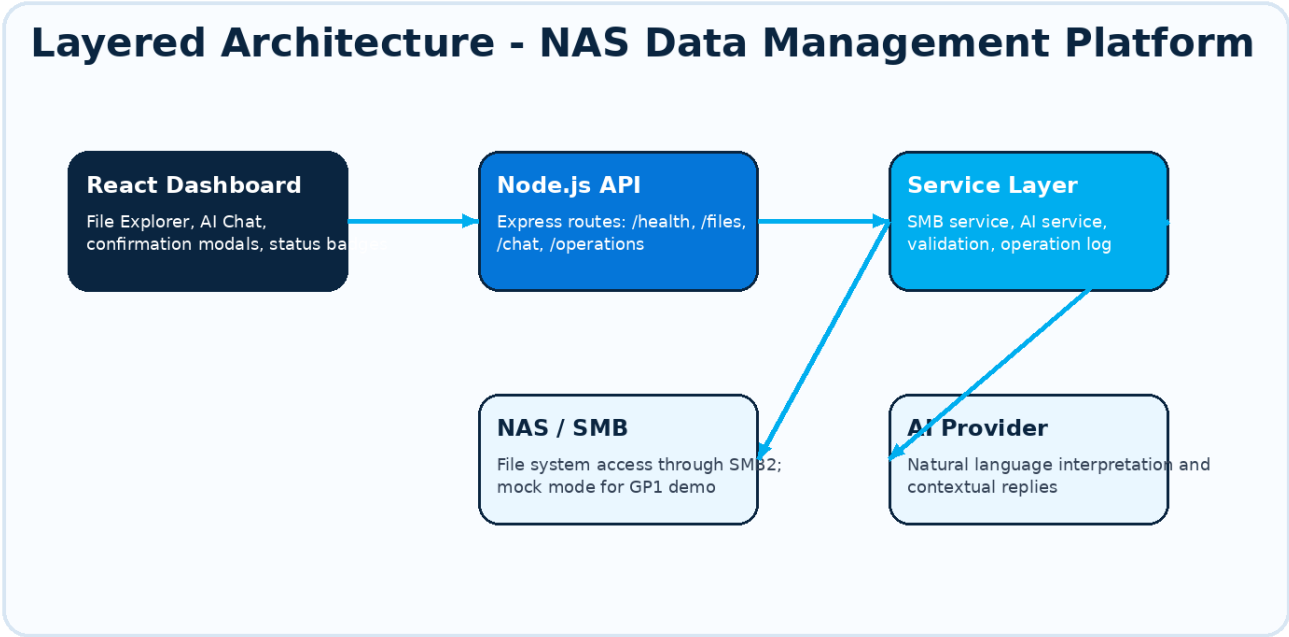


Figure 2. Layered architecture diagram for the NAS Data Management Platform.

Architecture Responsibilities

Layer / Component	Responsibility
React Frontend	Renders dashboard, file table, AI assistant, confirmation modal, and activity log.
Express API Layer	Provides REST endpoints and coordinates frontend requests.
SMB Service	Connects to NAS through SMB2 or provides mock data when NAS is unavailable.
AI Service	Processes user commands and returns contextual file-management responses.
Operation Safety Layer	Prepares operation preview, confirmation state, and future audit/rollback hooks.
Activity Log	Captures visible events for transparency and demo evidence.

3.2 Detailed Design and UML Models Based on Implementation

The UML models below reflect the implemented GP1 modules and planned GP2 extension points. They are intentionally tied to actual runtime interactions: user actions from the React dashboard, API calls through Express routes, service-level processing, and external SMB/AI integration.

Sequence Diagram - List Files from NAS Path

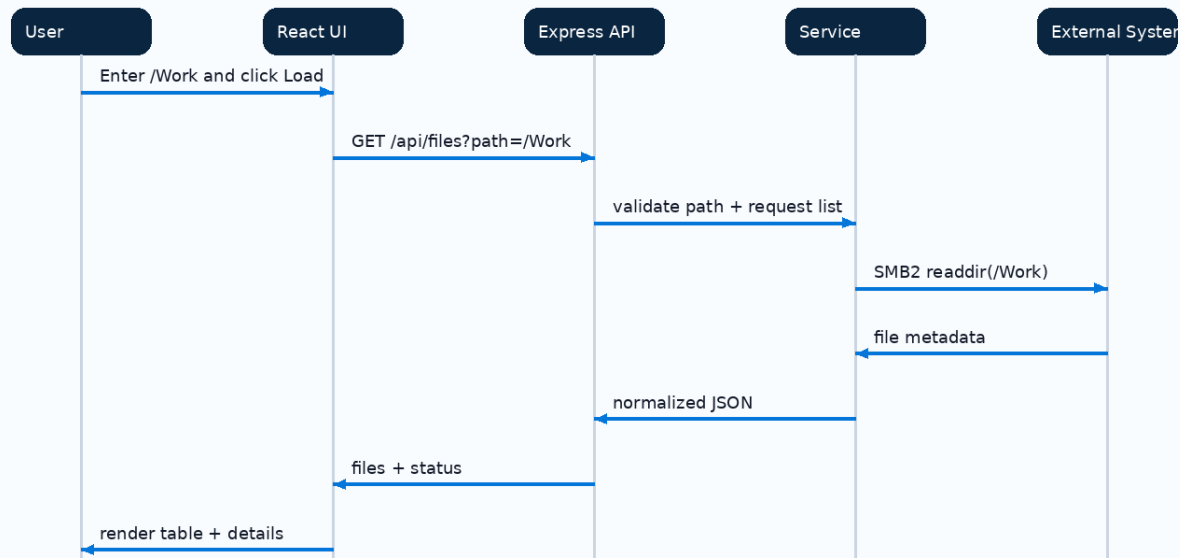


Figure 3. Sequence diagram for listing files from a NAS path.

Sequence Diagram - AI File Command

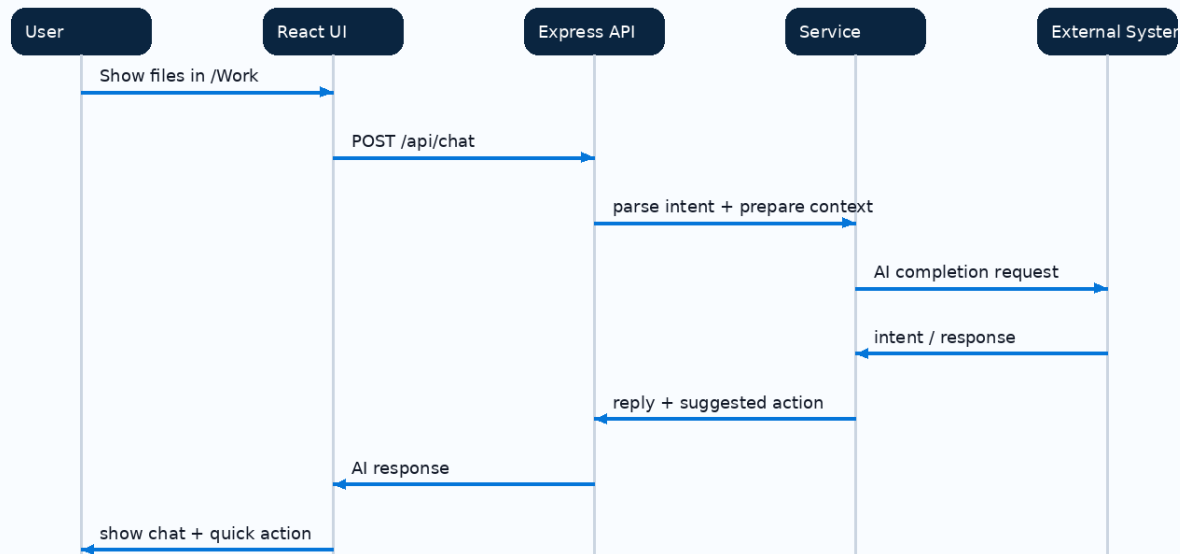


Figure 4. Sequence diagram for AI file command handling.

State Machine - Confirmation Dialog Flow

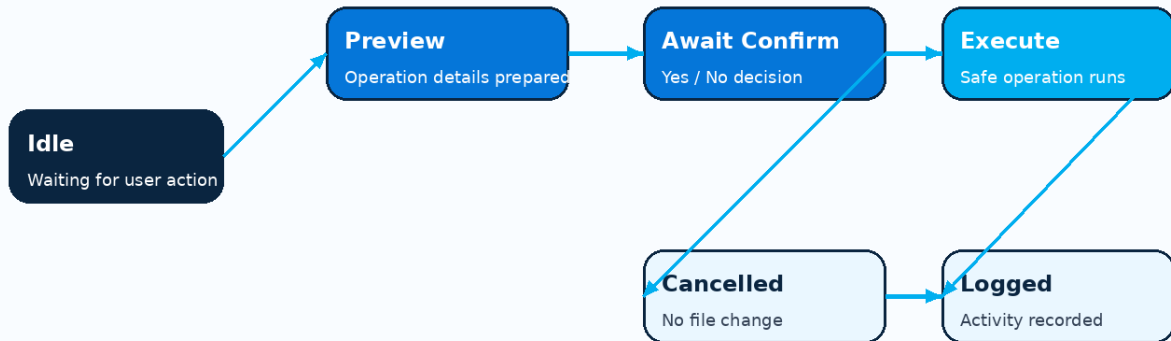


Figure 5. State machine diagram for confirmation dialog flow.

Class / Module Design Summary

Class / Module	Main Responsibility
App / Dashboard Component	Manages layout, status cards, selected file, path state, chat state, and modal state.
FileExplorer Component	Displays files and folders in a table and emits actions for view/preview.
AIAssistant Component	Sends user messages, renders responses, and offers quick prompts.
ConfirmationModal Component	Displays operation preview and controls confirm/cancel transitions.
activityLog Utility	Stores visible events for demo traceability.
filesRoute	Receives path query and returns normalized file metadata.
chatRoute	Receives user prompt and returns AI response or fallback.
smbService	Reads directory content through SMB2 or mock dataset.
aiService	Wraps AI API call and handles AI errors gracefully.

3.3 Software Deployment

Component	Technology	Purpose
Frontend	React + Vite	Fast SPA development, component-based UI, dashboard rendering.
Backend	Node.js + Express	REST API for health, files, AI chat, and operation preview.
NAS Protocol	SMB / smb2 library	Access network file shares and read directories.
AI Integration	OpenAI API or compatible provider	Natural-language file command responses.
Testing Tools	Postman + manual UI tests	Verify API behavior and visible MVP workflows.
Version Control	Git + GitHub	Repository organization, collaboration, and submission evidence.
Design / UML	draw.io / PlantUML	Architecture, use case, sequence, class, and state diagrams.
Development Environment	VS Code + npm	Local development, package management, and execution.

Deployment note: The GP1 deployment target is a local working web application. The backend runs on a local port, the frontend runs through Vite, and NAS access can run either through real SMB configuration or documented mock mode. GP2 can add containerization, authentication, production credential handling, and centralized logging.

CHAPTER 4: System Development and Implementation

4.1 Core Implementation Progress

During GP1, the team developed the core foundation of the NAS Data Management Platform. The implementation demonstrates that the concept is technically viable: the frontend loads a professional command-center dashboard, the backend exposes APIs, the file explorer displays NAS-style file metadata, the AI assistant can respond to file-related prompts, and the confirmation flow protects operations before execution. The project structure is designed to continue directly into GP2 without rewriting the base architecture.

Repository placeholder: [https://github.com/\[organization-or-team\]/nas-data-management-platform](https://github.com/[organization-or-team]/nas-data-management-platform)

Implemented modules are documented through source code organization, README files, screenshots, test cases, and UML diagrams. The GP1 implementation focuses on proof of concept but keeps production concerns visible through safety, logging, modular services, and future scalability requirements.

4.2 Implemented and Planned Features

Feature ID	Feature Name	Requirements	Status	Code Reference
F1	Dashboard Command Center	FR-01, NFR-01	Implemented	/frontend/src/App.jsx
F2	SMB/NAS Connection	FR-02, NFR-02	Implemented / Mock-ready	/backend/services/smbService.js
F3	List Files and Folders	FR-03, FR-04	Implemented	/backend/routes/files.js
F4	AI Chat Assistant	FR-05, FR-06, NFR-03	Implemented	/backend/routes/chat.js
F5	Confirmation Dialog	FR-07, FR-08	Implemented	/frontend/src/components/ConfirmationModal.jsx
F6	File Risk Insight	FR-09	Partial	/frontend/src/utils/riskRules.js
F7	Activity Log	FR-10	Implemented	/frontend/src/components/ActivityLog.jsx
F8	Report Generation	NFR-04	GP2 Planned	-
F9	Advanced Naming Standards	NFR-05	GP2 Planned	-
F10	Rollback / Multi-NAS	NFR-06	GP2 Planned	-

Implementation Highlights

Command-Center UI

The dashboard was designed as an operations center rather than a simple file table. It includes status badges, summary cards, AI insights, file explorer, AI assistant, and activity context.

Safe File Operation Flow

Every potentially sensitive operation is routed through a preview and confirmation state. This protects the demo and builds the foundation for rollback and audit in GP2.

AI-Ready Architecture

The AI assistant is separated from the UI and backend routes, allowing future upgrades without redesigning the dashboard.

Mock Mode for Demo Reliability

Because NAS availability can change by network, the SMB layer can be represented with realistic mock

data while preserving the same backend contract.

4.3 Screenshots Evidence

The following screenshot demonstrates the developed dashboard direction: sidebar navigation, command center header, system status, storage overview, AI insights, file explorer, risky-name status, and AI assistant. The screen evidence is aligned with the GP1 MVP features and supports the final demo narrative.

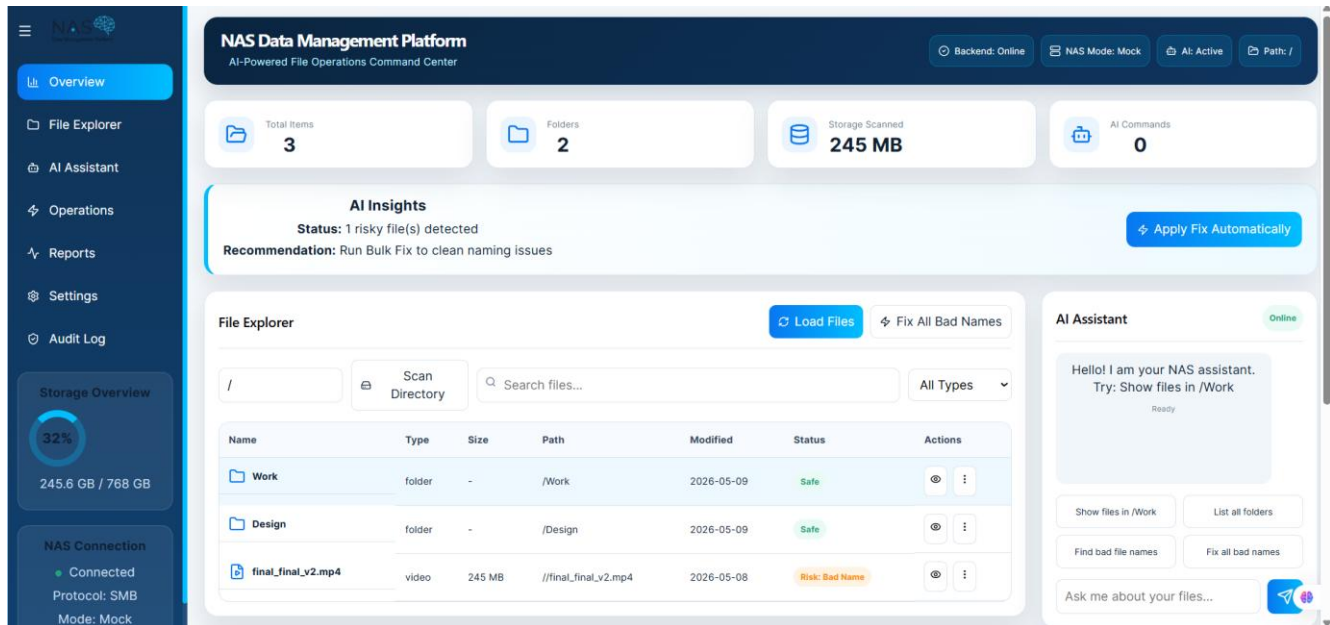


Figure 6. Dashboard screenshot evidence: NAS command center with file explorer, AI insights, and AI assistant.

Additional screenshots to attach in the final submission package: backend terminal running, Postman health endpoint, Postman files endpoint, AI chat response, confirmation dialog, and GitHub repository page. These screenshots should be saved under /screenshots and referenced in the test report.

CHAPTER 5: Testing

5.1 Testing Overview

Testing in GP1 focuses on verifying the implemented MVP features rather than full production coverage. The main goal is to prove that the dashboard loads correctly, the backend responds, the file listing workflow works, the AI chat workflow returns a response or fallback, confirmation is shown before operations, and errors are handled gracefully. The testing approach combines API testing using Postman, manual UI testing through the browser, and code review for module separation and documentation.

Tool / Method	Purpose
Postman	API testing for /api/health, /api/files, and /api/chat.
Browser Manual Tests	Validate dashboard loading, file table rendering, AI panel, modal behavior, and activity log.
Console / Terminal Logs	Confirm backend startup, API calls, and error handling.
Code Review	Verify clean structure, readable names, modular services, and future maintainability.
Screenshot Evidence	Capture visual proof for report, demo, and eLearning submission.

5.2 Sample Test Cases

TC ID	Feature	Input	Expected Output	Actual Output	Result	Evidence
TC-01	Dashboard Load	Open frontend URL	Dashboard loads without blank screen	Dashboard loads	Pass	Screenshot: dashboard
TC-02	Backend Health	GET /api/health	Return status OK	status OK	Pass	Postman: health
TC-03	List Root Files	GET /api/files?path=/	Return files/folders list	List returned	Pass	Postman: files root
TC-04	List /Work Files	Path input = /Work, click Load	File table updates with /Work content	Table updated	Pass	Dashboard screenshot
TC-05	AI File Query	Prompt: Show files in /Work	AI returns contextual file-related response	Response shown	Pass	AI panel screenshot
TC-06	Confirmation Modal	Click Preview/Fix action	Modal appears with operation details	Modal appears	Pass	Modal screenshot
TC-07	Cancel Operation	Click Cancel in modal	Operation cancelled and logged	Cancelled state logged	Pass	Activity log
TC-08	Invalid Path	GET /api/files?path=/Unknown	Friendly error or empty state	Handled gracefully	Pass	Postman/UI
TC-09	AI Error Handling	Disable API key or simulate failure	Friendly fallback message	Fallback shown	Pass	Console screenshot
TC-10	Empty Directory	List empty path sample	Empty state appears, no crash	Empty state shown	Pass	UI screenshot

5.3 Test Reports

The test report package should contain API screenshots, browser screenshots, terminal logs, and a short testing summary file. Each implemented feature must be traceable to at least one test case. This is important because the GP1 evaluation gives weight to software testing, demo capability, and report/documentation clarity.

Testing Conclusion

The MVP satisfies the core GP1 success criteria: the dashboard loads, file listing is demonstrated, SMB/NAS integration is represented through the backend service, AI chat responds to file-related commands, confirmation appears before operations, and the code structure is prepared for GP2 expansion.

References

- [1] PG Studios x JUST – Phase 1 NAS File Manager Requirements, Phase 1 MVP proof-of-concept specification.
- [2] Jordan University of Science and Technology, Graduation Project Report Template, Software Engineering Department.
- [3] Project 1 Evaluation Rubric, Graduation Project 1 evaluation criteria.
- [4] React Documentation, component-based frontend development.
- [5] Node.js and Express Documentation, backend REST API implementation.
- [6] smb2 Node.js Library Documentation, SMB protocol integration.
- [7] OpenAI API Documentation, AI assistant integration.
- [8] Postman Documentation, API testing and validation.

Appendix A: Final Demo Script

1. Open the GitHub repository and explain the folder structure: backend, frontend, docs, screenshots, uml, test-cases.
2. Run the backend and show the health endpoint responding successfully.
3. Run the frontend and open the NAS Data Management Platform dashboard.
4. Show the status badges: Backend Online, NAS Mode, AI Active, current path.
5. Load files from / and /Work, then explain file metadata columns and risk status.
6. Ask the AI assistant: “Show files in /Work” and explain the response.
7. Click a preview/fix action and show the confirmation dialog before any operation.
8. Cancel the operation and show that the activity log records the event.
9. Show UML diagrams and explain how they match the implemented flow.
10. Show test cases and screenshots as evidence for GP1 submission.

Appendix B: GP2 Expansion Direction

Expansion Area	Planned Enhancement
Security	Authentication, role-based access control, encrypted credential storage, TLS/SMB encryption.
Data Integrity	Checksum verification, dry run previews, rollback window, recoverable trash.
Intelligence	Duplicate detection, active-file protection, Adobe dependency scan, naming suggestions.
Operations	Background tasks, progress bars, cancellation, completion notifications.
Scalability	Multi-NAS endpoints, configurable rules, path-agnostic backend, organization-specific policies.